

JOHN CARROLL UNIVERSITY
Department of Math, Computer Sci, & Data Sci

From Perceptron to MLP

Handwritten Digit Recognition on MNIST

A comparison of a Single-Layer Perceptron and a Multilayer Perceptron across learning rates and hidden-layer architectures

Sava Crnjak

CS/DATA 3440-51 · Machine Learning · Project 1
Fall 2026 · Submitted 09/22/2026

Contents

1. Introduction	3
2. Dataset and Data Preparation	3
3. Single-Layer Perceptron	4
4. Multilayer Perceptron	4
5. Experimental Results	5
6. Model Evaluation	6
7. SLP vs. MLP Discussion	9
8. Conclusion	9
<i>Appendix: AI Usage</i>	10

Headline Results

Model	Architecture	Training Accuracy (%)	Test Accuracy (%)
SLP (learning rate 0.001)	784 → 10	79.63	89.47
MLP 3 (best on unseen data)	784 → 128 → 10	99.98	97.96

1. Introduction

This project modifies the Single-Layer Perceptron (SLP) developed in class to the MNIST handwritten digit dataset and compares it against the Multilayer Perceptron (MLP). The task is a ten-class classification problem: given the 784 pixel values of a 28 x 28 grayscale image, predict which digit, 0 through 9, was written.

Apart from accuracy, the primary objective of the project was to investigate two design decisions and report what they change. First, the learning rate of the SLP is varied across 0.001, 0.01 and 1.0 while every other setting is held fixed, to observe its effect on the training process and on final performance. Second, the hidden-layer architecture of the MLP is varied across one hidden layer of 32, 64, and 128 neurons and a two-layer 64-32 configuration, to observe how added capacity affects both training accuracy and generalization to unseen images. The two model families are then compared directly using the same train/test data, with confusion analysis and an inspection of individual correct and incorrect predictions.

2. Dataset and Data Preparation

The MNIST dataset was loaded with `fetch_openml("mnist_784", version=1, as_frame=False)`. The dataset was examined before any training was carried out:

Table 1. MNIST dataset summary.

Property	Value
Number of examples	70,000
Number of features	784 (28 x 28 pixels)
Possible target values	10 classes (0-9)
Range of pixel values	0-255 (grayscale intensity)

Each image is flattened into a vector of 784 input features, so the classification problem maps 784 inputs onto 10 possible output classes. The pixel values were normalized into the 0-1 range by dividing the feature matrix by 256, which keeps the weighted sums in a manageable range and makes the size of each weight update proportional to pixel intensity rather than to raw scale.

The data was then split with `train_test_split` using 80% for training and 20% for testing, with `random_state=42` and stratification on the target so that all ten digits stay proportionally represented in both sets. That gives 56,000 training images and 14,000 test images. The identical split was reused for every model reported here, so all comparisons between learning rates, between architectures, and between the SLP and the MLP are made on exactly the same data.

3. Single-Layer Perceptron

The classroom SLP was written for 8 x 8 images, giving 64 input features. Adapting it to MNIST required changing the input dimension to 784, so the architecture becomes $784 \rightarrow 10$: a weight matrix W of shape (10, 784) and a bias vector of length 10, both initialized from a uniform distribution on $[0, 1]$ under a fixed `np.random.seed(42)`.

The step-by-step learning process from class was kept intact. For each training pattern the target is one-hot encoded (the digit 3 becomes $[0\ 0\ 0\ 1\ 0\ 0\ 0\ 0\ 0]$), the weighted sum $z = Wx + b$ is computed, a step function converts z into a binary output vector, and the error δ is the difference between target and output. Weights and bias are then updated by the learning rate times that error, and a pattern is counted as an error whenever any element of δ is non-zero. At test time the predicted digit is the neuron with the largest weighted sum.

Training settings

Table 2. SLP training configuration. Only the learning rate varies between runs.

Setting	Value
Architecture	$784 \rightarrow 10$ (no hidden layer)
Epochs	10
Learning rates tested	0.001, 0.01, 1.0
Weight initialization	Uniform $[0, 1]$, seed 42

Because the seed, the split and the epoch count were all held constant, any difference between the three runs is attributable to the learning rate alone.

4. Multilayer Perceptron

The MLP was built with `MLPClassifier` from `sklearn.neural_network`, trained on the same training data and evaluated on the same test data as the SLP. Each model used `max_iter=100` and `random_state=42`, so once again the hidden-layer architecture is the only variable that changes between runs.

Four architectures were trained and evaluated: three single-hidden-layer networks of increasing width, and one deeper network with two hidden layers. The single-layer widths of 32, 64 and 128 isolate the effect of capacity in one layer, while the 64-32 network tests whether splitting a comparable number of neurons across two layers helps or hurts. Training and test accuracy were obtained with the `.score` method on the training and test sets respectively.

Table 3. MLP architectures investigated.

Model	Hidden Layer(s)	Architecture
MLP 1	32	784 → 32 → 10
MLP 2	64	784 → 64 → 10
MLP 3	128	784 → 128 → 10
MLP 4	64, 32	784 → 64 → 32 → 10

5. Experimental Results

5.1 SLP learning-rate experiment

Table 4. SLP performance by learning rate. Epochs, seed and train/test split held constant.

Learning Rate	Epochs	Training Accuracy (%)	Test Accuracy (%)
0.001	10	79.63	89.47
0.01	10	79.47	86.58
1.0	10	79.32	85.89

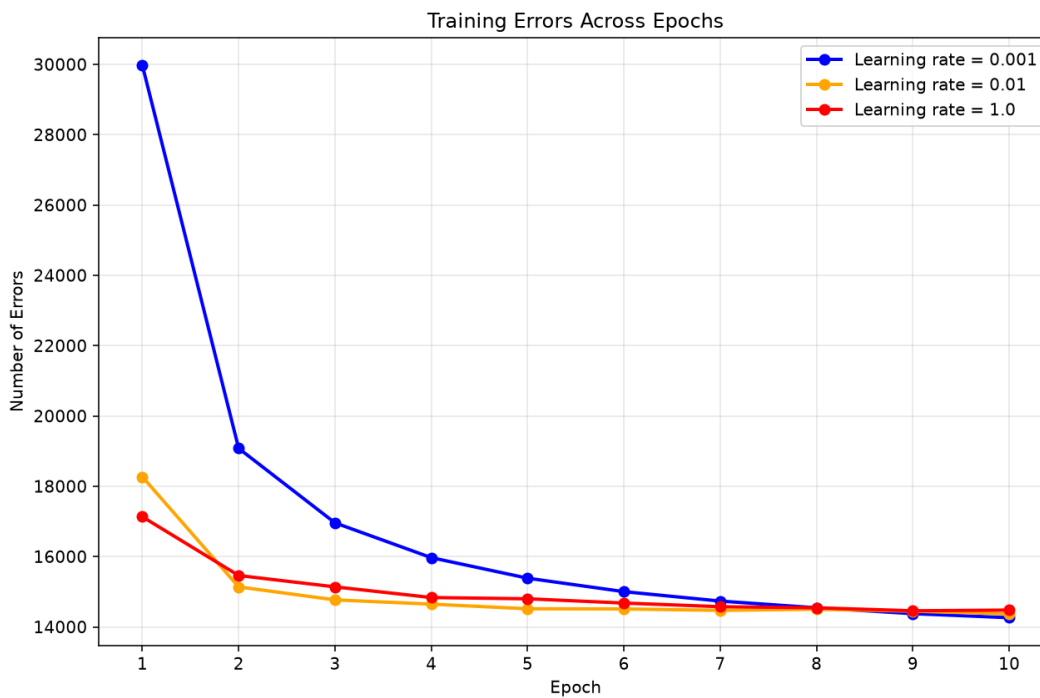


Figure 1. Number of training patterns with errors across epochs, by learning rate.

5.2 Observations on the learning rate

Did changing the learning rate affect the training process?

Changing the learning rate did slightly affect the training process. The higher learning rates began with an initial higher accuracy, but after the first 7 epochs, the 3 rates converged to an equal training accuracy around 79.2%. Past epochs 7 and 8, the lowest learning rate, 0.001, was the most accurate by a slight margin.

Did it affect the final training or test accuracy?

Yes, lower learning rates resulted in slightly higher training and test accuracies.

Which learning rate produced the best results, if any?

Although by a small margin, learning rate 0.001 produced the most accurate results. However, it was only more accurate after 8 epochs.

If the results were similar or unchanged, what did you observe about the learning process?

For this study, lower learning rates result in higher errors for the first few epochs, but eventually yield higher accuracies than higher learning rates, whose accuracy plateaus after the first few epochs.

5.3 SLP vs. MLP architectures

Table 5. Training and test accuracy for the SLP and all four MLP architectures. MLP 3 is the best-performing model on unseen data.

Model	Hidden Layer(s)	Architecture	Training Accuracy (%)	Test Accuracy (%)
SLP	None	784 → 10	79.63	89.47
MLP 1	32	784 → 32 → 10	99.99	96.50
MLP 2	64	784 → 64 → 10	100.00	97.61
MLP 3	128	784 → 128 → 10	99.98	97.96
MLP 4	64, 32	784 → 64 → 32 → 10	100.00	97.52

The best-performing MLP on unseen data is **MLP 3**, the 784 → 128 → 10 network, with a test accuracy of 97.96%. It is carried forward as the MLP in the evaluation and comparison that follows.

6. Model Evaluation

Confusion matrices were computed for both models with `confusion_matrix` over the same 14,000 test images, using the SLP trained at learning rate 0.001 and the 784 → 128 → 10 MLP. For each matrix, the number of times a digit was predicted incorrectly was obtained by summing its row and excluding the diagonal entry. The three most frequently misclassified digits for each model are reported below.

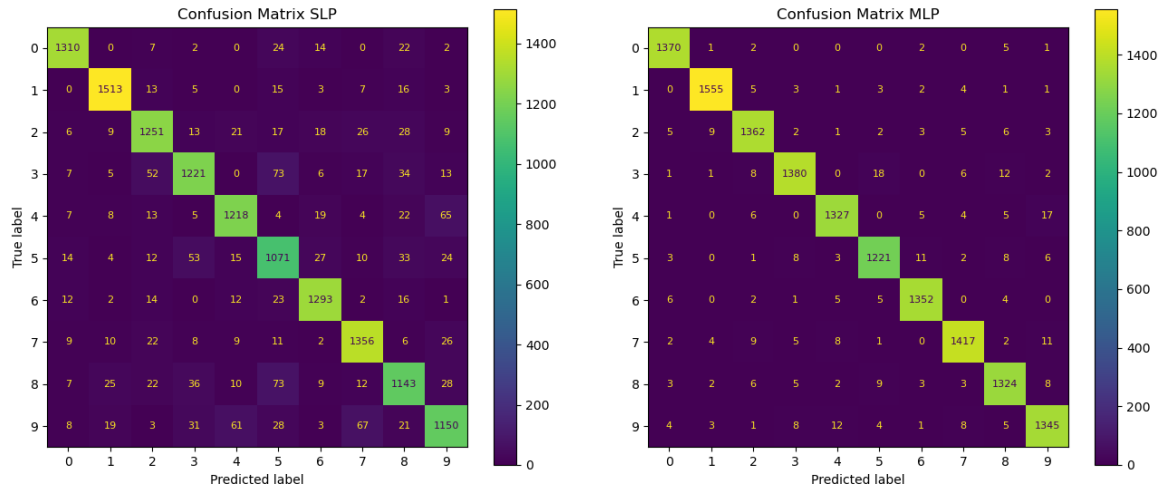


Figure 2. Confusion matrices over the 14,000 test images: the SLP at learning rate 0.001 (left) and the $784 \rightarrow 128 \rightarrow 10$ MLP (right). Rows are the true digit, columns the predicted digit, so every off-diagonal cell is a mistake.

Most Confused Digits (SLP)

Digit	9	8	3
Times Confused	241	222	207

Most Confused Digits (MLP)

Digit	3	9	5
Times Confused	48	46	42

Table 6. Three most frequently misclassified digits per model, counted as off-diagonal row totals of each confusion matrix.

6.1 Digit analysis

Which digits are most frequently confused?

3 and 9 were both in the top 3 most confused digits for both the SLP with learning rate 0.001 and the MLP with one hidden layer with size 128.

Does the SLP struggle with particular digits?

Yes, 9, 8, 3, and 5 were all digits it struggled with heavily. The MLP model greatly reduced the number of false predictions from mid 200s to the 40-50 range for those digits.

Does the MLP improve the classification of those digits?

Yes. Those digits are still the most difficult to classify for the MLP, but the overall classification accuracy has significantly increased with those digits.

Is there a noticeable difference between training and test accuracy?

For the SLP, yes. The training accuracy was 79.63%, which resulted in a test accuracy of 89.47%, whereas for the MLP model, the training accuracy was 99.98 % with a test accuracy of 97.96%.

6.2 Individual predictions

Six images were drawn from the test set and classified by the $784 \rightarrow 128 \rightarrow 10$ MLP. Each image is shown with its actual class and the class the model predicted.

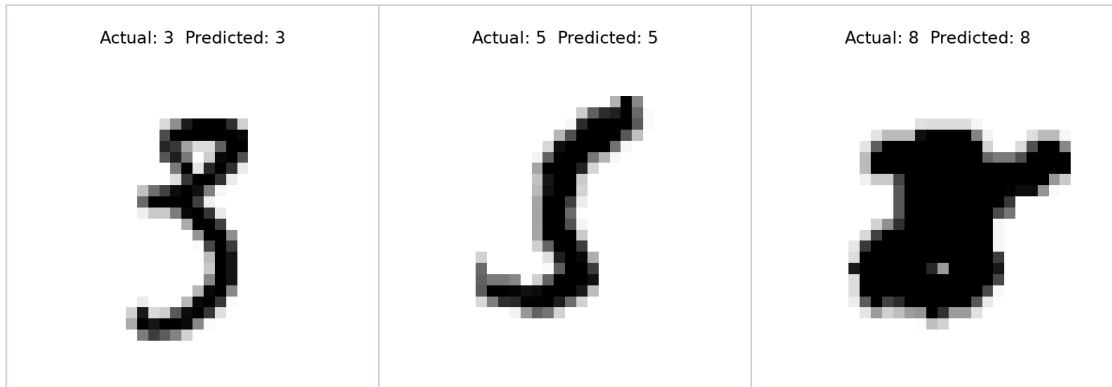


Figure 3. Three correctly classified test images.

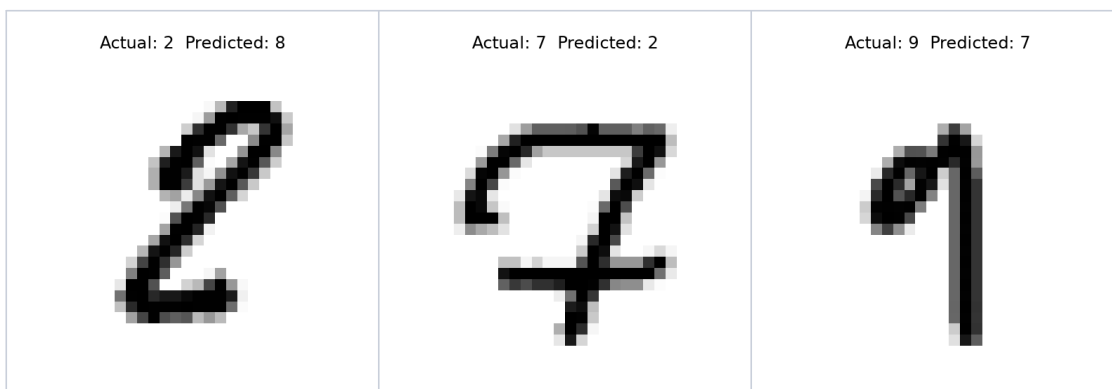


Figure 4. Three incorrectly classified test images.

Discussion of the incorrect predictions

Relies on unique parts of numbers to classify that number. However, when the numbers are written with different handwriting or style, certain features may resonate with a different number to which the feature is more common to.

It is possible that the model associates the pattern of thick, central clusters of pixels with the digit 8 as the cluster often forms in the middle of the 8 where the lines intersect. However, many 3s and 5s were confused with the number 8, as they also may have a similar central thickness, which may trick the model into classifying the digit as an 8.

9s, 7s, and 1s were also confused in cases where someone writes the number 1 with a slant at the top, or if someone writes a 9 with a thin oval instead of circle, both have been classified as 7s on many occasions, since the slant line pattern is universally more common to a 7 than it is to a 1 or 9, which can be drawn without the resembling pattern.

7. SLP vs. MLP Discussion

Network Architecture: How is the architecture of the MLP different from the SLP?

The MLP has a hidden layer of 128, which takes in the learned weights from the 784 pixels, and learns them once more to develop more accurate weights to be condensed into 10 digits.

Hidden Layers: What role do hidden layers play?

Hidden layers play the same role as the neurons in the SLP, except that instead of taking in the raw data, the hidden layer assesses the output from the first layer of neurons to fine-tune the weights even further, allowing for more complex pattern recognition from the model, which can lead to higher accuracies. In this case, the hidden layer size refers to the number of neurons that will each assess all 784 pixels with different weights to discover patterns that can be used to classify digits, such as curves, strokes, and angles.

Performance: Which model achieved better test performance? How large was the difference?

The MLP yielded better test accuracy (97.96%) than the SLP (89.47%).

Model Complexity: How did changing the number of hidden neurons and adding a second hidden layer affect model performance?

All models with a hidden layer performed better, with the worst one resulting in a test accuracy of 96.50%. Adding two hidden layers (64,32) only performed better than the model with a hidden layer size of 32, but not the models with sizes 64 and 128.

Generalization: Compare training and testing performance. What does this tell you about how well the models generalize to unseen images?

The MLP models tested a bit worse than their training accuracy, whereas the SLP's testing accuracy was always better than their training accuracy. This shows that although the MLP is overall more accurate, it can overfit the training data at times, whereas the SLP model performs better on unseen data, showing that it does not overfit the training data.

Limitations: What limitations did you observe in the SLP? How did the MLP address some of these limitations?

The SLP could not match the MLP in accuracy, as the MLP improved the accuracy of all digit predictions. The SLP was more easily confused between common features of digits with similar structures, whereas it took sloppy handwriting to confuse the MLP models.

8. Conclusion

Across the three SLP runs, the learning rate changed the shape of the training curve more than it changed the endpoint of training. The smallest rate, 0.001, carried the most errors through the first seven epochs yet finished with the best training accuracy (79.63%), while the largest rate, 1.0, fell fastest and then flattened from the fourth epoch onward. By epoch 8 all three runs sat at roughly the same training accuracy. The clearer separation appeared on unseen data, where lower learning rates produced higher test accuracy: 89.47% at 0.001 against 85.89% at 1.0.

Adding a hidden layer mattered far more than tuning the learning rate. Every MLP architecture beat the SLP on the test set, with the weakest of them still reaching 96.50%. The best performer on unseen data was the $784 \rightarrow 128 \rightarrow 10$ network at 97.96%, roughly eight and a half percentage points above the SLP. Widening a single hidden layer helped steadily from 32 to 64 to 128 neurons, whereas splitting capacity across two layers (64, 32) landed below both the 64 and 128 single-layer networks, so extra depth by itself was not the deciding factor.

The two model families also failed in different ways. The SLP's test accuracy was consistently higher than its training accuracy and it did not overfit, but it was easily confused between digits that share structural features; 9, 8, 3, 5 as well as 9, 7, 1 accounted for its heaviest error counts. The MLP reversed that pattern, training almost perfectly and testing slightly lower, which shows it can overfit, while cutting false predictions on those same digits from the mid 200s down to the 40-50 range. Notably, 3 and 9 remained among the top three most confused digits for both models: the hidden layer reduced the severity of those confusions without eliminating their cause, which the individual predictions trace back to handwriting styles or quality whose strokes genuinely resemble another digit.

Appendix: AI Usage

- Incorporating lists/dictionaries to store statistics for the model comparison tables and line plot.
- Using the ".score" function to get the training and test accuracy's for Part 2.
- Developing the pdf layout of the final report.

All written analysis in this report is original. The wording appears exactly as it was drafted in the accompanying file `Insights and Analysis.txt`, which may be consulted for word-for-word reference.